

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»**

Отчёт по лабораторной работе
«Типовая работа по математическому анализу №1»

Выполнил работу:
Косарев И. А.
Академическая группа:
№ J3110

Принято
Табиева А. В.

Санкт-Петербург
2025

Оглавление

Введение	2
1.1 Цели и задачи	2
1.1.1 Задачи	2
Аналитический этап	3
2.1 Верхняя и нижняя суммы Дарбу	3
2.2 Критерий Римана интегрируемости функции	3
2.3 Интегралы Дарбу	3
2.4 Доп. достаточное условие интегрируемости	4
2.5 Сравнение значения с формулой Ньютона-Лейбница	4
Практический этап	5
3.1 Реализация	5
3.2 Код реализации	5
3.3 Анализ и построение графиков	9
Выводы	10

Введение

1.1 Цели и задачи

Цель: исследование интегрируемости функции $f(x) = e^x$ на промежутке $[0; 1]$.

1.1.1 Задачи

- Выполнить анализ функции: монотонность, ограниченность, интегрируемость
- Реализовать алгоритм взятия определенного интеграла
- Вычислить отклонение от истинного значения для каждого способа вычисления интеграла и сравнить

Аналитический этап

2.1 Верхняя и нижняя суммы Дарбу

Предварительный анализ

Заметим, что $f'(x) = (e^x)' = e^x > 0 \forall x \in [0; 1]$, что означает, что $f(x)$ возрастает на всем отрезке. Это значит, что $M_i = \sup(x_i, x_{i+1})_f = f(x_{i+1})$, а $m_i = \inf(x_i, x_{i+1})_f = f(x_i)$. Тогда верхняя и нижняя суммы Дарбу соответственно будут иметь следующий вид (берем конгруэнтное разбиение отрезка на n частей):

$$U(f, t) = \sum_{i=0}^{n-1} \Delta x_i M_i = \sum_{i=0}^{n-1} \frac{1}{n} e^{x_{i+1}} = \sum_{i=0}^{n-1} \frac{1}{n} e^{\frac{i+1}{n}}$$
$$L(f, t) = \sum_{i=0}^{n-1} \Delta x_i m_i = \sum_{i=0}^{n-1} \frac{1}{n} e^{x_i} = \sum_{i=0}^{n-1} \frac{1}{n} e^{\frac{i}{n}}$$

Вычисление сумм

Вычислим обе суммы по формуле суммы геометрической прогрессии:

$$U(f, t) = \frac{1}{n} \sum_{i=0}^{n-1} e^{\frac{1}{n}} (e^{\frac{1}{n}})^i = \frac{1}{n} \frac{e^{\frac{1}{n}} (1 - e^{\frac{1}{n}n})}{1 - e^{\frac{1}{n}}}$$
$$L(f, t) = \frac{1}{n} \sum_{i=0}^{n-1} (e^{\frac{1}{n}})^i = \frac{1}{n} \frac{(1 - e^{\frac{1}{n}n})}{1 - e^{\frac{1}{n}}}$$

2.2 Критерий Римана интегрируемости функции

Определение критерия:

$$f(x) \in R([a, b]) \Leftrightarrow \forall \epsilon > 0 \exists \tau : U(f, \tau) - L(f, \tau) < \epsilon$$

Рассмотрим:

$$U(f, \tau) - L(f, \tau) = (e^{\frac{1}{n}} - 1) \frac{1}{n} \frac{(1 - e)}{1 - e^{\frac{1}{n}}} = \frac{1}{n} (e - 1) < \epsilon$$

Возьмем $n = \lceil \frac{e-1}{\epsilon} \rceil + 1$. При данном n неравенство выполняется, значит, $f(x) = e^x \in R([0, 1])$.

2.3 Интегралы Дарбу

Рассмотрим (для устремления ранга к нулю берем конгруэнтное разбиение):

$$U(f) = \lim_{\lambda(\tau) \rightarrow 0} U(f, \tau) = \lim_{n \rightarrow \infty} \frac{\frac{1}{n} e^{\frac{1}{n}} (1 - e)}{1 - e^{\frac{1}{n}}} = \left[\frac{0}{0} \right] = \lim_{n \rightarrow \infty} (e-1) \frac{\frac{1}{n^2} e^{\frac{1}{n}} + \frac{1}{n^3} e^{\frac{1}{n}}}{e^{\frac{1}{n}} \frac{1}{n^2}} = \lim_{n \rightarrow \infty} (e-1) \left(1 + \frac{1}{n}\right) = e-1$$

$$L(f) = \lim_{\lambda(\tau) \rightarrow 0} L(f, \tau) = \lim_{n \rightarrow \infty} \frac{\frac{1}{n}(1-e)}{1 - e^{\frac{1}{n}}} = \left[\frac{0}{0} \right] = \lim_{n \rightarrow \infty} \frac{(e-1)^{\frac{1}{n^2}}}{e^{\frac{1}{n} - \frac{1}{n^2}}} = \lim_{n \rightarrow \infty} (e-1)e^{-\frac{1}{n}} = e-1$$

Как можно заметить, интегралы совпадают, что также является подтверждением $f(x) = e^x \in R([0, 1])$.

2.4 Доп. достаточное условие интегрируемости

Для дополнительной проверки интегрируемости функции $f(x)$ можно воспользоваться критерием Лебега (или же следствием из него). $f(x) = e^x$, как говорилось ранее, монотонна на всем промежутке, из чего следует, что

$$\forall x \in [0, 1] \quad f(0) = 1 \leq e^x \leq e = f(1)$$

Значит, функция ограничена и монотонна $\rightarrow$ по следствию из критерия Лебега функция интегрируема.

2.5 Сравнение значения с формулой Ньютона-Лейбница

Вычислим неопределенный интеграл от $f(x)$:

$$\int f(x) dx = e^x + C$$

Вычислим определенный интеграл по формуле Ньютона-Лейбница:

$$\int_0^1 f(x) dx = e^x \Big|_0^1 = e - 1$$

Как можно заметить, значение совпало с предыдущими вычислениями.

Практический этап

3.1 Реализация

Были написаны алгоритмы вычисления определенного интеграла:

- Методом прямоугольников (левые, правые, середина)
- Методом трапеций
- Методом Симпсона

Также были реализованы вспомогательные функции для конгруэнтного и рандомизированного разбиения отрезка и для визуализации сравнения точности всех методов (принцип работы описан в теле функций).

3.2 Код реализации

```
import numpy as np
import typing as tp
from random import uniform
import matplotlib.pyplot as plt

def congruent_part(a: float = 0, b: float = 1, n: int = 4) -> tp.Generator[float, None, None]:
    """Построить конгруэнтное разбиение отрезка

    Args:
        a (float, optional): начало отрезка. Defaults to 0.
        b (float, optional): конец отрезка. Defaults to 1.
        n (int, optional): кол-во разбиений отрезка. Defaults to 4.

    Raises:
        ValueError: в случае невалидного n

    Yields:
        Iterator[tp.Generator[float]]: итератор точек отрезка
    """
    if n <= 0:
        raise ValueError("Invalid num of partitions")

    gap = (b - a) / n
    curr = a

    c = 0
```

```

while c <= n:
    yield curr

    c += 1
    curr += gap

def rand_part(
    a: float = 0, b: float = 1, n: int = 4, lam: float = 0.025
) -> tp.Generator[float, None, None]:
    """Генератор рандомизированного разбиения отрезка

    Args:
        a (float, optional): Левая граница. Defaults to 0.
        b (float, optional): Правая граница. Defaults to 1.
        n (int, optional): Кол-во подотрезков. Defaults to 4.
        lam (float, optional): Параметр рандомизации (радиус сфер, в которых
        выбираются границы подотрезков). Defaults to 0.025.

    Yields:
        Iterator[tp.Generator[float, None, None]]: итератор границ подотрезка
    """

    if lam <= 0:
        raise ValueError("Invalid lam value")
    if n <= 0:
        raise ValueError("Invalid n value")

    if a > b:
        a, b = b, a

    gap = (b - a) / n
    curr = a
    lam = min((gap - 1e-8) / 2, lam)

    c = 0
    while c <= n:
        yield uniform(max(a, curr - lam), min(b, curr + lam))

        c += 1
        curr += gap

def rectangle_method(
    f: tp.Callable,
    mode: tp.Literal["L", "R", "M", "rand"] = "L",
    part_iter: tp.Generator = congruent_part(),
) -> float:
    """Посчитать интегральную сумму методом прямоугольников

    Args:
        f (tp.Callable): интегрируемая функция
        mode (tp.Literal[L, R, M, rand], optional): вид взятия значения функции в партии. Def
        part_iter (tp.Generator, optional): генератор границ подотрезков.

```

Raises:

ValueError: в случае невалидного mode

Returns:

float: интегральная сумма

```
"""
match (mode):
    case "L":
        choose_func = lambda x, y: x
    case "R":
        choose_func = lambda x, y: y
    case "M":
        choose_func = lambda x, y: (x + y) / 2
    case "rand":
        choose_func = lambda x, y: uniform(x, y)
    case _:
        raise ValueError("Invalid mode")

integ_sum = 0
x_r = next(part_iter)

for i in part_iter:
    x_l = x_r
    x_r = i

    integ_sum += (x_r - x_l) * f(choose_func(x_l, x_r))

return integ_sum
```

```
def trapezoid_method(f: tp.Callable, part_iter: tp.Generator = congruent_part()) -> float:
    """Посчитать интегральную сумму методом трапеций
```

Args:

f (tp.Callable): интегрируемая функция.

part_iter (tp.Generator, optional): генератор границ подотрезков.

Returns:

float: интегральная сумма

```
"""
integ_sum = 0
x_r = next(part_iter)

for i in part_iter:
    x_l = x_r
    x_r = i

    integ_sum += 0.5 * (x_r - x_l) * (f(x_l) + f(x_r))

return integ_sum
```

```
def simpson_method(f: tp.Callable, part_iter: tp.Generator = congruent_part()) -> float:
    """Посчитать интегральную сумму методом Симпсона
```



```

Args:
    f (tp.Callable): интегрируемая функция.
    part_iter (tp.Generator, optional): генератор границ подотрезков.

Returns:
    float: интегральная сумма
    """

integ_sum = 0
x_r = next(part_iter)

for i in part_iter:
    x_l = x_r
    x_r = i

    integ_sum += (x_r - x_l) * (f(x_l) + f(x_r) + 4 * f((x_l + x_r) / 2)) / 6

return integ_sum

def plot_comparison(f: tp.Callable, a: float = 0, b: float = 1, n: int = 4, acc: int = 100):
    """Изобразить сравнения точности вычисления интеграла разными способами

    Args:
        f (tp.Callable): интегрируемая функция
        a (float, optional): левая граница. Defaults to 0.
        b (float, optional): правая граница. Defaults to 1.
        n (int, optional): кол-во подотрезков в разбиении. Defaults to 4.

    Returns:
        plt.fig, plt.ax: фигуры графиков
        """

    part_iter = congruent_part(a, b, n)
    xs = np.linspace(a, b, n * acc)

    ys = [f(x) for x in xs]

    fig, ax = plt.subplots(1, 4, figsize=(15, 6))

    ax[0].plot(xs, ys, label="f(x)", color="black")
    ax[0].set_title("Метод прямоугольников (левые)")
    ax[0].set_xlabel("x")
    ax[0].set_ylabel("y")

    ax[1].plot(xs, ys, label="f(x)", color="black")
    ax[1].set_title("Метод прямоугольников (середина)")
    ax[1].set_xlabel("x")
    ax[1].set_ylabel("y")

    ax[2].plot(xs, ys, label="f(x)", color="black")
    ax[2].set_title("Метод трапеций")
    ax[2].set_xlabel("x")

```

```

ax[2].set_ylabel("y")

ax[3].plot(xs, ys, label="f(x)", color="black")
ax[3].set_title("Метод Симпсона")
ax[3].set_xlabel("x")
ax[3].set_ylabel("y")

x_r = next(part_iter)

for i in part_iter:
    x_l = x_r
    x_r = i
    x_m = (x_l + x_r) / 2

    ax[0].fill_between([x_l, x_r], [f(x_l), f(x_l)], color="purple", alpha=0.3)
    ax[1].fill_between([x_l, x_r], [f(x_m), f(x_m)], color="orange", alpha=0.3)
    ax[2].fill_between([x_l, x_r], [f(x_l), f(x_r)], color="blue", alpha=0.3)

    x_simp = np.linspace(x_l, x_r, 100)
    y_approx = np.polyval(np.polyfit([x_l, x_m, x_r], [f(x_l), f(x_m), f(x_r)], 2), x_simp)

    ax[3].fill_between(x_simp, y_approx, color="green", alpha=0.3)

ax[0].legend()
ax[1].legend()
ax[2].legend()
ax[3].legend()

return fig, ax

```

3.3 Анализ и построение графиков

Графики и их построение приведены в файле Jupyter Notebook, сохраненном в [репозитории лабораторной](#). Ниже приведен график сравнения ошибки разных методов интегрирования:

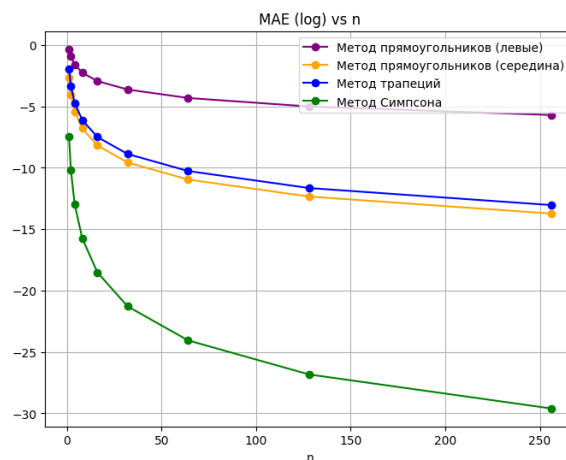


Рис. 3.1: Сравнение MAE (логарифмированной) в зависимости от точности интегрирования (колла подотрезков)

За основу вычисления ошибки был взят принцип МАЕ. Как видно на графике, самым точным методом оказался метод Симпсона. Можно заметить, что при увеличении кол-ва подотрезков интегрирования ошибка для всех методов уменьшается, что говорит о прямой корреляции точности и кол-ва подотрезков (ранга разбиения).

Выводы

В процессе выполнения лабораторной работы был выполнен анализ функции $f(x) = e^x$, проверка ее свойств. Также был написан простейший алгоритм интегрирования разными методами. Программа позволяет выбрать метод взятия границ подотрезков и их количество, что позволяет регулировать точность вычисления.

Также было визуализировано сравнение точности каждого из методов и проведен анализ данного графика.

В процессе выполнения лабораторной работы я смог получше разобраться с вычислением интегральных сумм и смог на практике применить свои теоретические знания.